

Document number: P3376R0
Date: 2024-10-14
Audience: SG21, EWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Contract assertions versus static analysis and ‘safety’

This paper tries to systematize the discussion regarding how contract annotations address ‘safety’ and how they interact with ‘static analysis’. We describe two different definitions of ‘safety’, and describe different kinds of ‘static analysis’. Depending on which definition one adopts, one draws different conclusions as to what the contract annotations should do.

We also review, in the context of safety and static analysis, the two visions for contract annotations:

1. Predicates are just C++ code, modulo the treatment of *unqualified-ids* as `const`, as proposed in [\[P2900R9\]](#).
2. Predicates are UB-free, side-effect-free *strict* expressions (with an escape hatch to switch to "just C++ code"), as proposed in [\[P2680R1\]](#) and [\[P3285R0\]](#), and argued for in [\[P3362R0\]](#).

1. The perception of safety

As a human, a consumer and a user of systems, tools and mechanisms, the question that is important to me is: *is this safe*? By which I mean, can this cause harm to me, to other people, to my privacy, my savings, my environment? In response to this, we have the notion of *system safety*, which, as [\[P3297R1\]](#) reminds us, is a practice of performing all possible tests at all possible integration levels, in order to be able to draw *statistical* conclusions as to how likely the system is to cause harm. The key aspects here are:

- it applies to the system as a whole,
- it is a statistical property.

This notion is very different from the notions like "UB safety" / "memory safety" / "exception safety" / "thread safety". It doesn't talk about "safe language constructs" or "safe languages". It is about the final product to satisfy certain properties. In pursuit of this ultimate goal, it is fine to employ a feature that has the *potential* to trigger UB (undefined behavior), as long as the system as a whole becomes less likely to cause harm.

On the other hand, we have the definition of a "safe operation" in a language, as formulated by David Abrahams in [\[ABRAHAMS22\]](#):

A safe operation is one that can't cause undefined behavior.

In this view, accessing any object via a reference of unknown origin is an unsafe operation:

```
bool has_name(string const& s) // reference may be dangling
{
    return !s.empty();         // may be accessing a non-object
}
```

UB- and memory-safe code can also be source of unsafe programs. Consider:

```
bool should_launch_missiles(
    bool enemy_launched_missiles,
```

```

    bool system_armed)
{
    return enemy_launched_missiles
        || system_armed; // bug: should be `&&`
}

```

It can also be shown that naively increasing UB-safety can decrease system safety. Consider:

```

bool alert()
{
    bool alerting_on = check_if_alerting_is_on();
    bool alert_user; // #1 uninitialized variable

    Alerts all_alerts = get_all_alerts(alert_user); // #2 typo: wrong variable passed
    alert_user = filter_by_user(all_alerts);
    log_state(alerting_on, alert_user);

    return alert_user;
}

```

This function has a bug. The intention was to pass variable `alerting_on` to function `get_all_alerts` in #2. As a consequence, when this function is invoked, it triggers UB in C++23, because variable `alert_user` is read prior to any value assignment in #2. If a compiler issues a warning about this, and a programmer naively applies the advice "always initialize your variables", they will assign any value in #1, and the warning will be silenced, advice will be followed, UB will be prevented, but a life-critical program will become incorrect, and therefore unsafe.

In contrast, when the programmer doesn't try to initialize the variable, and runs a tool like Clang-Tidy, the tool will highlight places #1 and #2 in the code, and the programmer will be drawn attention to the fact that a typo has occurred at #2, and will be able to pass the correct variable to the function. Thus, our goal cannot be "just don't have UB": it should be "have correct code". Program correctness is an important factor in system safety.

Undefined behavior is always a consequence of a bug. On the other hand, in consequence, it can trigger more bugs. Thus, a tool that (1) prevents UB and (2) does not conceal bugs is required. In this spirit, C++ has now the notion of *erroneous behavior* ([\[P2795R5\]](#)).

Bottom line: while there is an overlap between UB-safety and system safety, they are in general different goals.

2. Static analysis

How can a program analysis — static or dynamic — conclude that a program is correct, or incorrect? You measure the program correctness by comparing its behavior against the programmer intentions or against the program specification. One thing that helps here is undefined behavior: it is a safe assumption that it is never the programmer's intention to cause undefined behavior, so whenever we can demonstrate that a program hits or would hit UB, it is incorrect. We do not know which part exactly is incorrect, but we can be *sure* that at least one part is.

Similarly, contract annotations will also be able to help tools performing program correctness verification. This is because they can encode parts of the program specification. It is safe to assume that it is never the programmers intention to have any contract predicate evaluate to `false`.

There are different goals that static analysis tools can have. Here, we will describe only one, in two variants. This is where a tool, looking at a single translation unit, tries to answer the question if a given function `f` calls all functions

in its body *in contract*; that is, if all preconditions of the called functions evaluate to true.

```
bool pre(X const& x); // not defined in this translation unit

X make() post(r: pred(x));
void use(X const& x) pre(pred(x));

void f(X const& a, X const& b, X const& c)
  pre(pred(a))
{
  use(x);          // #1 OK, matches with f's precondition

  if (pred(b))
    use(b);        // #2 OK, matches with condition in `if`-statement

  use(make());     // #3 OK, matches with the postcondition of `make()`

  use(c);          // #4 WARNING: nothing to match with
}
```

In this form of analysis, we treat predicates in the preconditions of the called functions as *symbols*. That is, we do not try to determine what is evaluated inside them. Instead, if function `use()` has symbol `pred(x)` associated with its argument, we try to determine, if the same symbol has been produced by the preceding part of the function body. The call at #1 is ok because this symbol is already a precondition on `f()`. The call at #2 is ok, because `pred()` is already checked in the `if`-statement. The call at #3 is ok, because the same symbol is produced by the postcondition of `make()`. The call at #4 is not ok, because we cannot find symbol `pred(c)` in the scope of function `f()`, so a warning is produced.

In order for thus described analysis to give *certainty* of correctness, a number of conditions would have to be satisfied:

1. The code invoked in function `f()`, for instance the definition of function `use()`, does not have UB.
2. The definitions of contract annotations do not have UB.
3. The code invoked in function `f()`, for instance the definition of function `use()`, does not alter the state of the program in a way that changes the results of `pred()` at different points in the function execution.
4. The definition of `pred()`, doesn't alter the state of the program.

Because contract annotations are a new feature, we are not bound by backwards compatibility requirement and can afford defining it the way we need. So it is possible to enforce the assumptions #2 and #4 above. This is advocated for in [\[P3362R0\]](#).

An alternative, is to leave all four points as assumptions. This results in reduced confidence, but is still valuable. This would render the type of analysis similar to the one for axiom-kind predicates described in [\[P2176R0\]](#).

We will call these two form of verification *strict precondition matching* and *relaxed precondition matching*, respectively.

3. Why is [\[P2900R9\]](#) useful already

Contract annotations are a tool for communicating parts of program specification. They can be used for detecting bugs. Detecting and removing bugs increases system safety.

The primary value that [\[P2900R9\]](#) brings to the table is:

- the standardized notation for contract assertions and
- the position where you put them.

Precondition assertions are visible in the translation unit where the guarantee is delivered by the caller; postconditions are visible in the translation unit where the guarantee is consumed. This means that a static analysis task of verifying if preconditions of the called functions are satisfied by postconditions of other called functions can now be narrowed to a single translation unit, and is therefore more affordable in terms of resources.

The fact that the syntax for expressing contract annotations is standardized, enables different tools to recognize the syntax, and expect that third party libraries will use it. The tools can now perform static analysis on a new library code out of the box, rather than instructing the users to first put the tool-specific annotations. Library writers can use the standard notation for expressing the contract, rather than trying to figure out which tool they are going to target.

Precondition and postcondition assertions can now be placed in the right sequence between other operations. Currently, with assertion macros (which work as either expressions or statements), it is difficult to express that a precondition should hold before the constructor initializer list starts. Similarly, it is difficult to express a postcondition when a function has multiple return statements or when the expression in the `return`-statement produces an rvalue.

The contract predicates are regular C++ code, so they can be runtime-evaluated if needed. The value added to the runtime-checking of contract assertions is that, apart from calling them in the right places, you have a finer-grained control over which assertions are actually called. A library can be compiled in a no-runtime-checking mode, but the precondition runtime checks can still be added in the caller code. This is described in [\[P2780R0\]](#).

Next, the way precondition assertions are expected to help prevent bugs is not necessarily through runtime evaluation or static analysis, but because IDEs can simply display them in places where functions are called. Imagine that a programmer writes the following function call and gets the following hint from the compiler, where the precondition is displayed but with function argument names substituted for parameter names:

```
int min = config.read("min");
int max = config.read("max");
return is_in_range(val, min, max);
/*-----
|  PRE: val <= min      |
-----*/
```

The user expected the precondition to say `min <= max`, so they are alerted that something is wrong. They are now motivated to go and read the function declaration, which is:

```
bool is_in_range(int lo, int hi, int v)
    pre (lo <= hi);
```

And now the programmer learns that they put the arguments to the function in the wrong order. Bug prevented!

Further, because contract assertions are visible on function declarations, [\[P2900R9\]](#) enables the form of static analysis that we called the relaxed precondition matching. This analysis can be contained to a single translation unit. It doesn't give as certain correctness results as [\[P3362R0\]](#) would expect, but allows the predicates to be normal C++ code, and sufficient to detect bugs in programs, even if not performing *full* correctness proofs. Remember that system safety is a statistical property.

Finally, by the anecdotal evidence, a significant portion of the precondition and postcondition predicates are of the form:

```
i > 0           /* int comparisons */
p != nullptr
vec.empty()
vec.size() > i
x.has_value() /* inline fun that only returns the
               value of a boolean member data */
```

They are simple enough, using only inline functions, that the assumption of non-UB and no-side-effect often holds.

4. *Strict* assertions

Strict assertions, as described in [\[P2680R1\]](#) and [\[P3285R0\]](#), would guarantee zero modifications to the program state (outside the predicate), and zero UB. That is, if they were implementable. C++ is so saturated with *narrow-contract* constructs that trying to reject code with the UB potential effectively means preventing practically any code.

[\[P3285R0\]](#) from the outset gives up on preventing UB caused by data races. This is just too difficult. As the next thing, it will also need to give up on memory-safety-related UB, or alternatively narrow the applicability of strict predicates only to functions taking non-view parameters by value. More often than not, functions parameters are references. When such a function is invoked, the reference may already refer to a non-object: that is, be a dangling reference. However "safe" the internals of a strict predicate are, it already starts with a corrupt input. We could say that `std::object_address` has special powers where upon the function call, in the caller it tries to prove that the reference points to a valid object, and when this cannot be proven we get a compiler error. Let's picture a common case:

```
void foo(int const& i)
    pre(std::object_address(&i) && i > 0);

void bar(int const& i)
{
    if (i > 0)
        foo(i);
}
```

Inside function `bar` we cannot validate if the reference is non-dangling, because `bar` already receives a reference. This is a common case, and it would fail to compile. We would get a feature that is UB-safe but not applicable to the real code.

True, there are languages, that can guarantee zero memory issues, but this is because they are memory-safe all across. But here we are talking about a small 'safe' execution of a predicate after a long 'unsafe' execution of the program. This boundary between the 'unsafe' and the 'safe', corrupts the safety guarantee of the 'safe' part.

Next, given that *conveyor* functions do not affect overload resolution, strict predicates will behave uncomfortably in cases where a class offers a `const` and a non-`const` overload of a member function: a case very common in STL containers. The following small example illustrate this.

```
template <typename T>
class Wrapper
{
public:
```

```

        T& get() & conveyor;
    const T& get() const& conveyor;
};

```

And now we want to write a function with a strict precondition:

```

void modify(Wrapper<int> & w)
    pre(w.get() > 0);

```

This fails to compile because `w` is function argument and it would be passed to a function (`Wrapper::get`) taking the argument by a non-const reference, which is banned by [\[P3285R0\]](#). The author would have to change the predicate to:

```

void modify(Wrapper<int> & w)
    pre(std::as_const(w.get()) > 0);

```

This works, but forces us to type a different predicate than we intended, and this is likely to impede some forms of static analysis:

```

void work(Wrapper<int> & w)
{
    if (w.get() > 0) // <-- same predicate as
        modify(w);    // in the precondition?
}

```

Next, is it possible to implement function `std::vector::operator[]` as a conveyor? The implementation would probably look like this.

```

T const& vector::operator[](index_type i) const
{
    return _array[i];
}

```

In the return statement we have a postfix expression dealing with two runtime values, one of which is an address of a dynamically allocated array. This cannot pass as a conveyor, because it is impossible to prove that this access is valid without a difficult flow analysis. There would have to be a "disable conveyor safety" block that allows doing "unsafe" things inside the conveyor functions. But that would compromise the initial goals of guaranteeing no UB.

Next, while the idea of preventing UB in constrained environments (such as conveyor functions) is fine in general, some ways of achieving it, proposed in [\[P3285R0\]](#), are not acceptable, and could be themselves source of bugs, and therefore negatively contribute to system safety. For instance, the idea that the arithmetic overflow should produce an implementation-defined value, resulting in wrap-around arithmetic. The effect would be that programmers would start relying on the new semantics, and one could no longer detect whether some situations in code are intentional or not. Also, the same expressions would now render different values in precondition assertions and in function bodies:

```

int f (int i, int j)
    pre (i > 0)
    pre (j > 0)
    pre (j + i < 0)    // clever way to check if the combined values are big enough
{
    // under a wrap-around arithmetic
    if (i > 0)
        if (j > 0)
            if (j + i < 0) // skipped as either false or UB.

```

```

return 0;

    _Undefined_behavior(); // UB is hit.
}

f(INT_MAX, 2); // precondition passes, but in-body check fails

```

This particular aspect could be fixed by saying that arithmetic overflow is an erroneous behavior ([\[P2795R5\]](#)) and results in unspecified value. Alternatively, we could only allow addition provided that there is code in the close vicinity that proves that the addition will not overflow:

```

int f1 (int i, int j)
    pre (j + i > 0)                // Error: could overflow

int f2 (int i, int j)
    pre (std::add_sat(j, i) > 0) // OK, cannot overflow

int f3 (int i, int j)
    pre (-100 < i && i < 100)
    pre (-100 < j && j < 100)
    pre (j + i > 0)                // OK, we know max ranges

```

In summary, the design for strict contract assertions is not just buggy, it doesn't address fundamental issues. The advertised approach "let's start with something small and then expand" doesn't work here. The proposed small initial state is too small, and there is not even a sketch of the plan on how (if at all) it can be expanded. The present design for strict contract assertions doesn't give confidence that even if we postpone the adoption of contracts by additional three or six years, we would get something that fulfills the promise of usable UB-free and side-effect-free predicates.

The author's requirement on the viability of the "strict assertions" solutions are two. First, not substituting a seemingly well behavior for UB when a programmer plants a bug in the predicate; instead, substitute it for a diagnosable error. Second, the solution should allow the expression of the following types of predicates.

```

i > 0          /* int comparisons */
p != nullptr
vec.empty()
vec.size() > i
x.has_value() /* only returns the value of
               a boolean member data */

```

5. Discussion

[\[P3362R0\]](#), much like [\[P3285R0\]](#), argues that even when the strict contract assertions are present in C++, there would still be a need for *relaxed* contract assertions, which behave practically like assertions in [\[P2900R9\]](#). One could imagine the adoption order where relaxed predicates are included in C++26, and strict predicates are included in one of the next revisions of C++. However, [\[P3362R0\]](#) argues that releasing relaxed predicates before strict predicates would be a failure. This claim is made without providing sufficient rationale. How is "adding relaxed in C++26 and strict in C++29" worse from "adding both in C++29"? On the other hand, [\[P3297R1\]](#) argues that delaying the addition of contracts (even in [\[P2900R9\]](#) form) will damage the C++ community. We seem to have two conflicting future predictions.

One incompatibility of the "relaxed first, strict second" approach with the "strict and relaxed together" approach is about which of the two types of contract assertions is the default. The strict one would be a better default as this

would encourage users to choose this type, and therewith offer additional benefits of correctness checking. Would the change to [P2900R9] that requires the addition of contextual keyword `relaxed` after `pre`, `post` and `contract_assert` address the concerns?

6. Acknowledgments

Ville Voutilainen helped me understand the strict predicates better, and offered a critique of the paper which significantly improved its quality.

7. References

- [ABRAHAMS22] — David Abrahams, "Values: Safety, Regularity, Independence, and the Future of Programming", (<https://youtu.be/QthAU-t3PQ4?si=b6yz24Ps5B5DXG2R>).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P2176R0] — Andrzej Krzemiński, "A different take on inexpressible conditions" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2176r0.html>).
- [P2680R1] — Gabriel Dos Reis, "Contracts for C++: Prioritizing Safety", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2680r1.pdf>).
- [P2780R0] — Ville Voutilainen, "Caller-side precondition checking, and `Eval_and_throw`", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2780r0.html>).
- [P2795R5] — Thomas Köppe, "Erroneous behaviour for uninitialized reads", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2795r5.html>).
- [P2900R9] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++", (<https://isocpp.org/files/papers/P2900R9.pdf>).
- [P3023R1] — David Sankel, "C++ Should Be C++", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p3023r1.html>).
- [P3285R0] — Gabriel Dos Reis, "Contracts: Protecting The Protector", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3285r0.pdf>).
- [P3297R1] — Christian Eltschig, Mathias Kraus, Ryan McDougall, Pez Zarifian,, "C++26 Needs Contract Checking", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3297r1.pdf>).
- [P3362R0] — Ville Voutilainen, Richard Corden, "Static analysis and 'safety' of Contracts, P2900 vs. P2680/P3285", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3362r0.html>).